

Low-Cost Stochastic Hybrid Multiplier for Quantized Neural Networks

BINGZHE LI, University of Minnesota

M. HASSAN NAJAFI, University of Louisiana at Lafayette

DAVID J. LILJA, University of Minnesota

With increased interests of neural networks, hardware implementations of neural networks have been investigated. Researchers pursue low hardware cost by using different technologies such as stochastic computing (SC) and quantization. More specifically, the quantization is able to reduce total number of trained weights and results in low hardware cost. SC aims to lower hardware costs substantially by using simple gates instead of complex arithmetic operations. However, the advantages of both quantization and SC in neural networks are not well investigated. In this article, we propose a new stochastic multiplier with simple CMOS transistors called the *stochastic hybrid multiplier* for quantized neural networks. The new design uses the characteristic of quantized weights and tremendously reduces the hardware cost of neural networks. Experimental results indicate that our stochastic design achieves about 7.7x energy reduction compared to its counterpart binary implementation while maintaining slightly higher recognition error rates than the binary implementation. Compared to previous stochastic neural network implementations, our work derives at least 4x, 9x, and 10x reduction in terms of area, power, and energy, respectively.

CCS Concepts: • **Hardware** → *Arithmetic and datapath circuits; Application specific integrated circuits;*

Additional Key Words and Phrases: Stochastic computing, quantized neural network, multiplier, low power design

ACM Reference format:

Bingzhe Li, M. Hassan Najafi, and David J. Lilja. 2019. Low-Cost Stochastic Hybrid Multiplier for Quantized Neural Networks. *J. Emerg. Technol. Comput. Syst.* 15, 2, Article 18 (March 2019), 19 pages.

<https://doi.org/10.1145/3309882>

1 INTRODUCTION

Neural networks as a computational model based on neurons are becoming a prevalent method in many areas. Researchers start to investigate neural networks with hardware implementation rather than restricting to software implementations. In the hardware implementations, two major

This work was supported in part by National Science Foundation (NSF) grant no. CCF-1408123. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF. Preliminary portions of this work were presented in the 19th International Symposium on Quality Electronic Design (ISQED'18) [18].

Authors' addresses: B. Li and D. J. Lilja, University of Minnesota, Minneapolis, MN 55455; emails: {lix1743, lilja}@umn.edu; M. H. Najafi, University of Louisiana at Lafayette, Lafayette, LA 70503; email: najafi@louisiana.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

1550-4832/2019/03-ART18 \$15.00

<https://doi.org/10.1145/3309882>

directions are explored. One aims to accelerate neural network performance by using FPGA or VLSI designs [11, 27]. The other targets to low-power applications such as mobile devices and Internet-of-Things (IoT).

To achieve low hardware cost, quantization and stochastic computing (SC) have been investigated. SC [5] is used as a low-cost and fault-tolerant technology in approximate computation areas. By using simple gates to implement complex arithmetic operations, SC achieves extremely low hardware cost. For example, several finite state machines are able to implement exponential function or tanh function [22]. Thus, SC is a promising approach to reduce hardware cost in many applications. The quantization is an approximate method in many fields [6]. In terms of smaller quantization levels, quantization introduces larger errors while reducing complexity of designs significantly.

On one hand, previous works [13, 15, 17, 20, 23] have investigated SC-based neural networks. The stochastic restricted Boltzmann machine (RBM) implementation was proposed by Li et al. [16, 17]. Another work [12] implemented a tanh-based neuron and reduced error rates of stochastic neural network substantially. Two works [20, 23] implemented stochastic CNNs with approximate parallel counters (APCs) using different encoding formats to achieve low-cost design. On the other hand, prior work studied efficient hardware implementations of neural networks with the quantization technology. Hwang and Sung [9] studied fixed-point feedforward neural networks with different quantization levels. According to their results, they achieved similar recognition error rates compared to floating-point neural networks. Another work [34] exploited sparseness in vocabulary speech recognition to reduce the model size and execution time. However, the advantages of both quantization and SC in neural networks are not well investigated. Therefore, there is an opportunity to design a quantized neural network by using SC to further reduce hardware cost of neural networks.

In this article, we propose a new stochastic architecture for quantized neural networks. Regarding neural network implementations, first we retrain a fully trained neural network with the back-propagation algorithm to obtain neural networks with quantized weights. Second, according to the quantized weights, a stochastic matrix multiplication is implemented with a new component called the *stochastic hybrid multiplier* (SH-multiplier). The SH-multiplier is capable of efficiently summing up products of matrix elements and quantized weights. As a result, the SH-multiplier can decrease the number of inputs of parallel counters and thus reduce the hardware cost of the whole neural network.

The main contributions of this article are summarized as follows:

- Neural networks with different quantization levels (quantized weights) are implemented by SC.
- A stochastic quantized matrix multiplier is proposed with SH-multipliers for the quantized neural networks. This new approach efficiently achieves high accuracy in partial matrix multiplication by only using several AND and OR gates.
- The stochastic neural networks with the proposed multipliers achieve much lower hardware costs compared to previous stochastic implementations and compared to corresponding binary implementation counterparts while maintaining very close recognition error rates to the baseline binary neural networks.

The remainder of this article is organized as follows. Section 2 introduces the background of SC. Section 3 demonstrates the motivation of this work and discusses quantized neural networks. Section 4 provides the proposed SC-based neural networks. The experimental results and neural network comparisons are discussed in Section 5. The conclusions are given in Section 6.

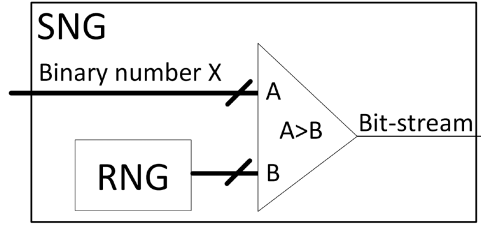


Fig. 1. An SNG consists of an RNG and a comparator.

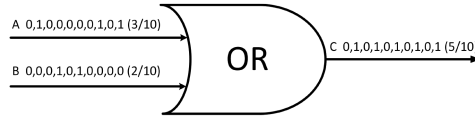


Fig. 2. OR gate for unipolar addition with unipolar coding.

2 STOCHASTIC COMPUTING

SC [2, 28] is a promising technology to achieve low-power and low-area designs. In the SC domain, a stochastic bit stream is a way to express a real value, which consists of a sequence of binary digits. The value expressed by a stochastic bit stream is computed from the primary statistic of the bit stream or the probability of any given bit in the stream being a logic '1.' Two types of encoding formats are used in representing stochastic bit streams: unipolar and bipolar [3]. These formats express a real number x in $[0, 1]$ or $[-1, 1]$, respectively. For the unipolar format, the probability of having 1's in a bit stream X is $P(X) = x$. However, for the bipolar format, the probability of seeing 1's in a bit stream X is $P(X) = (x + 1)/2$ [21]. Furthermore, to express values larger than 1 or smaller than -1 , scaled functions are first used to scale values into the range of $[-1, 1]$ or $[0, 1]$. Then, the scaled values can be represented by bit streams. The scaled functions can be found in Equations (1) and (2):

$$P(X) = x/N, \quad (1)$$

$$P(X) = \frac{x/N + 1}{2}, \quad (2)$$

where N is the scaling coefficient to map x/N to $[-1, 1]$ or $[0, 1]$.

2.1 Stochastic Number Generator

A stochastic number generator (SNG) (Figure 1) is used to generate bit streams for SC. It is conventionally implemented using random number generators (RNGs) and comparators. To implement the RNGs, linear feedback shifted registers (LFSRs) have been used with pseudo random property. Some SNGs [10, 24, 26, 33] have been proposed for high accuracy bit stream generation. In these SNGs, certain patterns are generated sequentially to increase the accuracy of bit streams with short stream lengths. Moreover, by using spintronic devices, low-cost structures have been introduced to reduce the hardware cost of SNGs and to achieve high performance at a reasonable power and area cost [12, 25].

Dickson et al. [4] and Qian and Riedel [29] introduced two methods of adding stochastic bit streams. Dickson et al. used a simple standard OR gate to perform approximate addition as illustrated in Figure 2. The example in this figure shows that the output of OR gate, 5/10, is exactly the desired result from adding input values, 2/10 and 3/10. According to Equation (3), performing an add operation using an OR gate introduces an extra AB term that is considered as an error to the

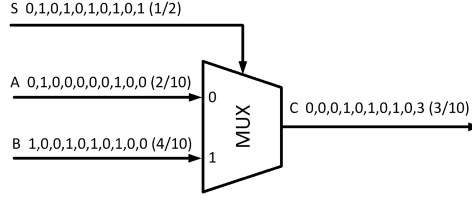


Fig. 3. MUX gate for scaled addition with unipolar bit streams.

result. The AB term can be ignored when $A \ll 1$ and $B \ll 1$.

$$C = A \text{ or } B = A + B - AB \quad (3)$$

Qian and Riedel implemented a “scaled addition” using a standard multiplexer (MUX) unit. According to Equation (4), when the select line of the MUX is a stochastic stream with a probability of 0.5, the normal add operation changes to $C = (A + B)/2$, which scales the results down by a factor of 2. Figure 3 shows an example of performing scaled addition.

$$C = A \cdot S + B \cdot (1 - S) \quad (4)$$

To generalize the simple 2-input adder to a multi-input adder, Equations (3) and (4) can be extended to Equations (5) and (6). In the ideal case for Equation (6), N can be any positive integer. However, if we are restricted to only use 2-input MUXs in the implementation of a multi-input adder, N should be a power of 2.

$$C = A_1 \text{ or } A_2 \text{ or } A_3 \text{ or } \dots \text{ or } A_N \quad (5)$$

$$C = \frac{1}{N} \sum_{i=1}^N A_i \quad (6)$$

In this work, we use unipolar encoding format throughout stochastic neural network implementations. Thus, AND gates and OR gates are used for the basic operations such as multiplication and addition in the stochastic neural networks. Moreover, another important part in neural networks is the activation function. We use the RBM with a sigmoid activation function [7]. Some previous works have investigated stochastic sigmoid implementation. For example, Li et al. [16] proposed a stochastic sigmoid function based on stochastic $\tanh()$ function using finite state machines. Another work [20] used APCs connecting finite-state machines to implement the sigmoid function. In this work, the sigmoid activation function is integrated into the neuron architecture with the multiplier. The details are discussed in Section 4.

3 MOTIVATION AND QUANTIZED NEURAL NETWORKS

3.1 Motivation

A neural network usually has thousands of weights, resulting in a long processing time within CPU/GPUs or high hardware cost for neural network accelerators. Prior efforts reduced the model size of deep neural networks by quantizing all the trained weights [9]. The quantized weights decreased the complexity of neural networks and led to a lower processing time with CPU/GPU software-based implementations and a lower hardware cost with conventional hardware-based designs.

Similar to binary domain implementations, an advantage of quantization in the stochastic domain is a reduction in the number of non-zero weights because near-zero weights are quantized to zero. As seen in Figure 4, a large portion of weights in neural networks is around zero. Therefore,

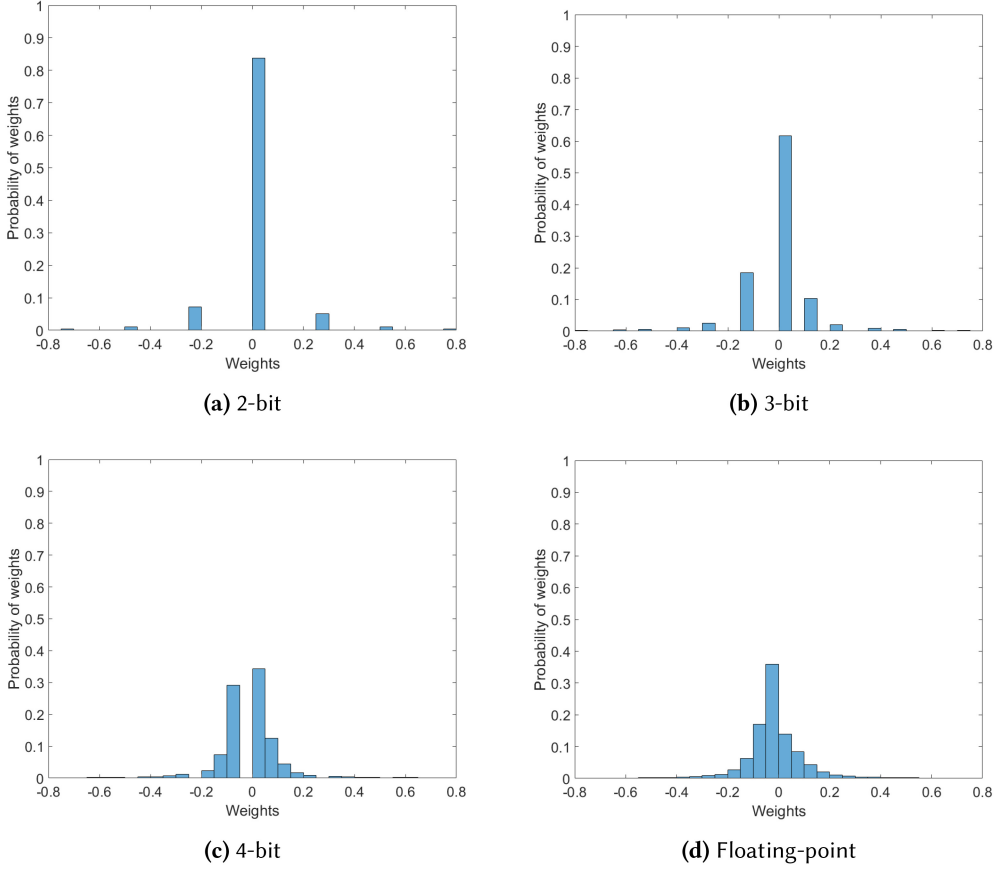


Fig. 4. The distribution of weights with the quantization levels of 2 bits, 3 bits, 4 bits, and original floating point (no retraining) for an RBM with the configuration of 784-500-1000-10.

removing near-zero weights can reduce a large number of operations. Another advantage is that the original weights are quantized to their nearby levels, and thus thousands of weights are replaced by several constant values. As a result, the quantized weights will further reduce resource utilization.

Directly applying SC to a quantized neural network may only obtain limited benefit. As the second advantage mentioned earlier, the benefit of quantized neural networks for stochastic implementations is reducing the number of SNGs. However, due to the correlation issue [1] between bit streams, we cannot encode the inputs by the same RNG in conventional stochastic implementations. Furthermore, low-cost SNGs have been investigated in prior work [13, 32]. As a result, the benefit of reducing the number of SNGs will become much smaller in the future. How to efficiently apply SC to the quantized neural networks motivates us to design a new stochastic neuron architecture.

3.2 Neural Network Retraining

In general, a neuron implements a function of $p = \sigma(\mathbf{A} * \mathbf{x} + \mathbf{B})$, where σ is the activation function and $\mathbf{x}$ is the input of the neuron, $\mathbf{A}$ is the coefficient for matrix multiplication, and $\mathbf{B}$ is the bias. Some previous works quantized weights ($\mathbf{A}$ or $\mathbf{B}$) or inputs ($\mathbf{x}$) of neural networks in the retraining

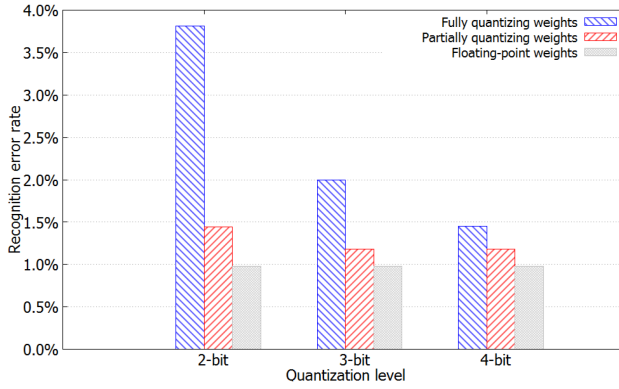


Fig. 5. Error rate comparisons between fully quantizing weights, partially quantizing weights, and floating-point weights for an RBM with the configuration of 784-500-1000-10.

process [6, 8, 9] to decrease the sizes of neural networks and thus reduce resource utilization. However, the trade-off is that the quantized neural networks typically increase error rates.

For stochastic quantized neural networks, we only quantize the coefficient **A**. The reasons are given as follows. First, our new proposed architecture obtains benefits from the the weight **A**, which is used for multiplications. Thus, it is not necessary to quantize inputs and other weights **B** in our implementations. Second, we use the same un-quantized input images as previous works so that our work is designed for the same input datasets as the previous works. Third, since quantization reduces precision of neural networks, quantizing unnecessary weights can result in increasing error rates. As seen in Figure 5, the results indicate that the neural network with partially quantized weights has better recognition error rates than the one with fully quantized weights. Therefore, to obtain higher recognition rates, we only quantize coefficient **A**.

In our training process, we use the following steps to retrain neural networks and obtain the quantized weights:

- (1) Fully train neural networks with floating-point weights.
- (2) Choose the number of training iterations to minimize the output error.
- (3) Quantize weights used for multiplications in hidden layers.
- (4) Remove near-zero biases of hidden layers.
- (5) Perform the back-propagation algorithm [30] to update weights.
- (6) Repeat Step 3 through Step 5 to minimize the output error and quantize partial weights.

After retraining the neural networks, we obtain the quantized neural networks, and all weights are regarded as constant values for stochastic neural network classifiers. We will show that compared to the floating-point neural network, the quantized neural network has slightly higher recognition error rates.

4 STOCHASTIC NEURAL NETWORK IMPLEMENTATIONS

In this section, we introduce stochastic implementations for quantized neural networks. We use 2-bit quantization as an example, and the architecture of the stochastic implementation is quite similar for other quantization levels.

4.1 Stochastic Quantized Bit Streams

In our stochastic implementation, multiplication for unipolar format bit streams is achieved by ANDing 2-input bit streams. One of the inputs is a quantized weight as a constant value. For 2-bit

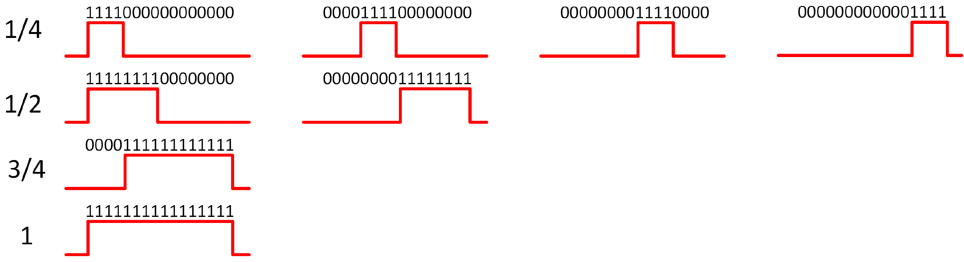


Fig. 6. Sequences of 2-bit quantized weights.

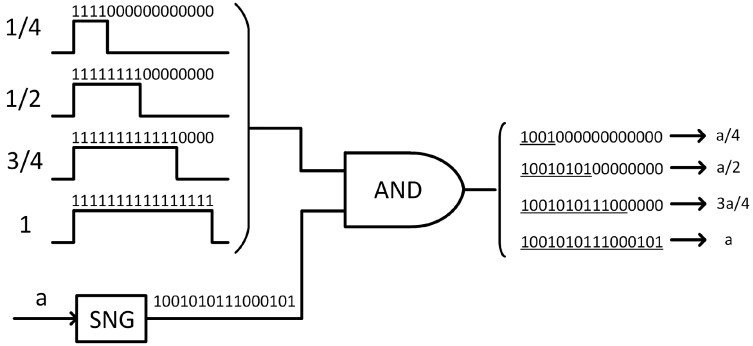


Fig. 7. An example of multiplication with 2-bit quantized weights.

quantized weights with unipolar format, the weights only have five values: 0, 0.25, 0.5, 0.75, and 1 in the stochastic domain. Thus, the multiplication is changed to selecting 0%, 25%, 50%, 75%, and 100% bits of the other input bit stream and then padding un-selected bits with zeros. As a result, only the selected bits contain useful information for future addition operations. Therefore, considering how to efficiently use the unselected bits, we propose a new type of bit stream for the quantized weights.

The quantized weight bit streams are similar to the time-encoded unary streams in previous works [10, 26]. The difference is that our quantized weight bit streams have continuous 1's located in different phases. An example of representing 2-bit quantized weights is shown in Figure 6. The quantized bit streams are generated from unary bit streams by shifting different bits. By using the quantized weight bit streams, the multiplication is performed by selecting a sequential part of a bit stream as illustrated in Figure 7. Finally, the output of quantized multiplication consists of a sequential part of one input and a sequence of zeros. The advantage of this type of encoding method will be discussed in the following sections.

4.2 Stochastic Quantized Multiplication and Addition

As discussed earlier, the multiplication with 2-bit quantized weights truncates inputs with a number of sequential bits and pads them with zeros. Therefore, if the output bit stream of the multiplication is padded with another product of two bit streams instead of padding zeros, the output bit stream will become the sum of two products (suppose that two product bit streams have enough padded zeros). For example, suppose that an operation is to compute $a/4 + b/4 + c/4 + d/4$ (which is indicated by $(1/4, 1/4, 1/4, 1/4)$). As seen in Figure 8, four bit streams (a , b , c , and d) generated by four SNGs are ANDed with four shifted unary bit streams of $1/4$ whose phases of 1's are interleaved. Those four bit streams of $1/4$ select a different part of a , b , c , and d . Consequently, after going

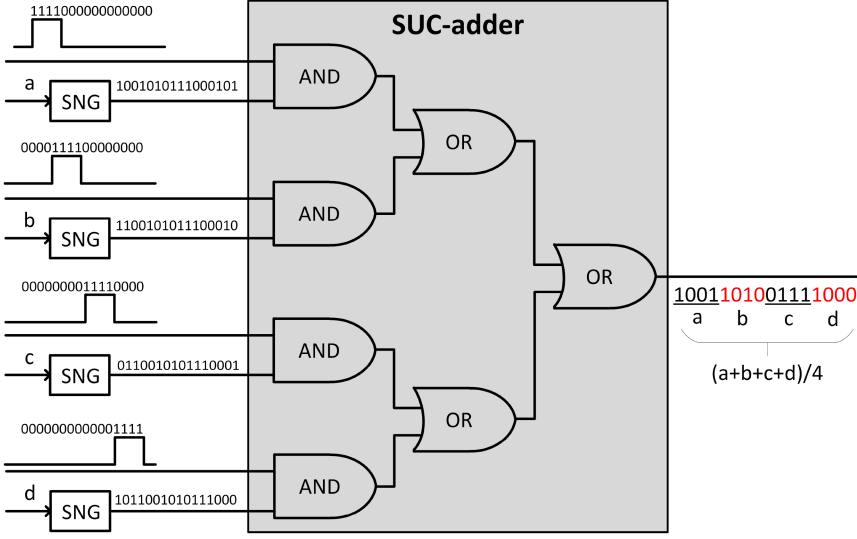


Fig. 8. An example of a 4-input SUC-adder with 2-bit quantized weights.

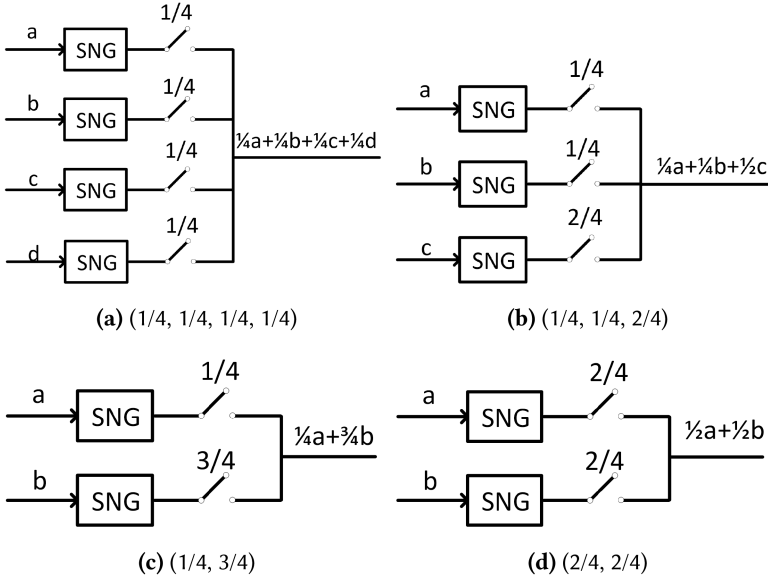


Fig. 9. SUC-adders with different combinations of weights. All quantized weight bit streams have different phases of 1's in each circuit. The four combinations are $(1/4, 1/4, 1/4, 1/4)$, $(1/4, 1/4, 2/4)$, $(1/4, 3/4)$, and $(2/4, 2/4)$.

through OR gates, the output becomes $(a + b + c + d)/4$. Our prior work [18] proposed an adder called the *stochastic unary code adder* (SUC-adder), whose inputs are products of shifted unary bit streams and conventional random stochastic bit streams. The SUC-adder uses only seven simple gates in the preceding example to sum up those four products. For different combinations of quantized weights, the SUC-adders are implemented by different circuits. Figure 9 lists four types of SUC-adders for 2-bit quantized weights. For stochastic quantized neural network implementation,

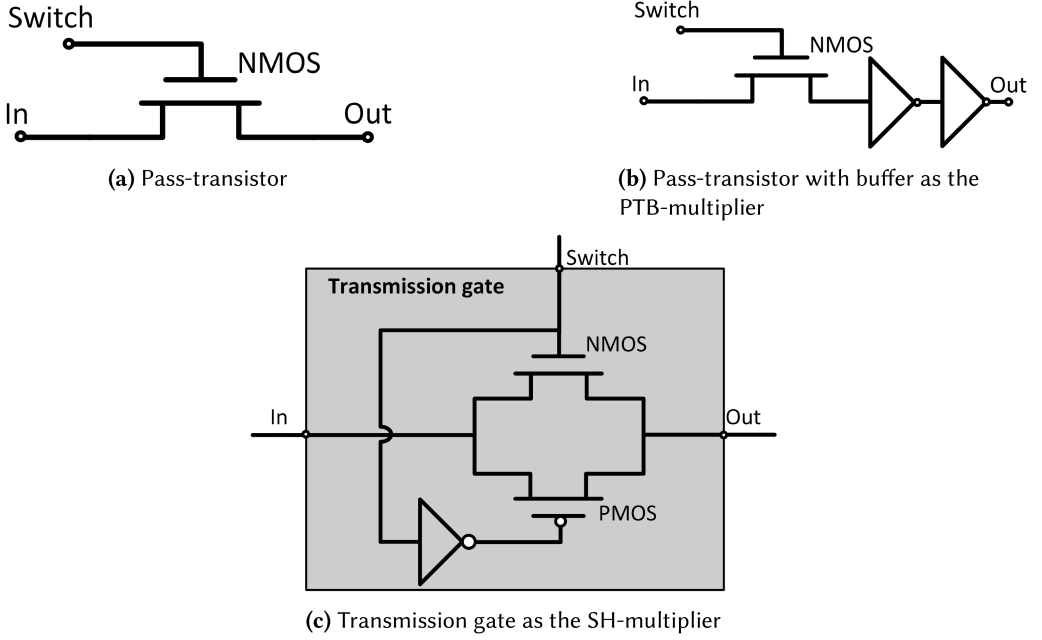


Fig. 10. The schematics of different switches.

since 2-bit quantized weights are constant values and we know the total number of each quantized value, the number and types of SUC-adders are determined after training the neural networks. For each SUC-adder, we make sure that the sum of all coefficient of the adder is 1.

In this work, to simplify the SUC-adder, we propose a new partial matrix multiplier, called the *stochastic hybrid multiplier* (SH-multiplier). The proposed multiplier uses several CMOS devices as a switch to select different phases of bit streams. The simplest form of a switch is an NMOS as a pass-transistor [35]. Figure 10(a) shows the schematic of an NMOS pass-transistor. An NMOS device faces an issue to pass the high-level value or 1 since the output only charges up to $V_{dd} - V_{th}$ due to the body effect. The signal weakens as it passes to the next stages of logic. A similar issue happens for the PMOS pass-transistor, which is not effective at passing 0. A solution to this issue is to add two inverters as a buffer after the pass-transistor to charge the voltage back to V_{dd} for the NMOS case as seen in Figure 10(b). (We call the multiplier consisting of the pass-transistors with buffers the *PTB-multiplier*). However, using two inverters as a buffer results in additional hardware power and area cost overhead. To make the switch as simple as possible while still effectively passing correct voltages to next stages, we use the transmission gate (see Figure 10(c)) as the switch to select different phases of bit streams. The transmission gate can effectively pass 0 via NMOS and pass 1 via PMOS with a complementary structure. Compared to our prior work [18] as shown in Figure 8, using the transmission gate as a switch can accomplish the quantized multiplication instead of using an AND gate. We call the architecture (Figure 11) consisting of transmission gates the *SH-multiplier*. By combining different phases of bit streams together, the SH-multiplier can directly use wire connection to achieve exactly the same summation results as using OR gates.

Moreover, the hardware cost between SUC-adder, PTB-multiplier, and SH-multiplier is compared in terms of 3-bit quantized multipliers. There are seven types of inputs with numbers from 2 to 8, and 21 combinations of quantized weights. For example, 2-input architecture with 3-bit

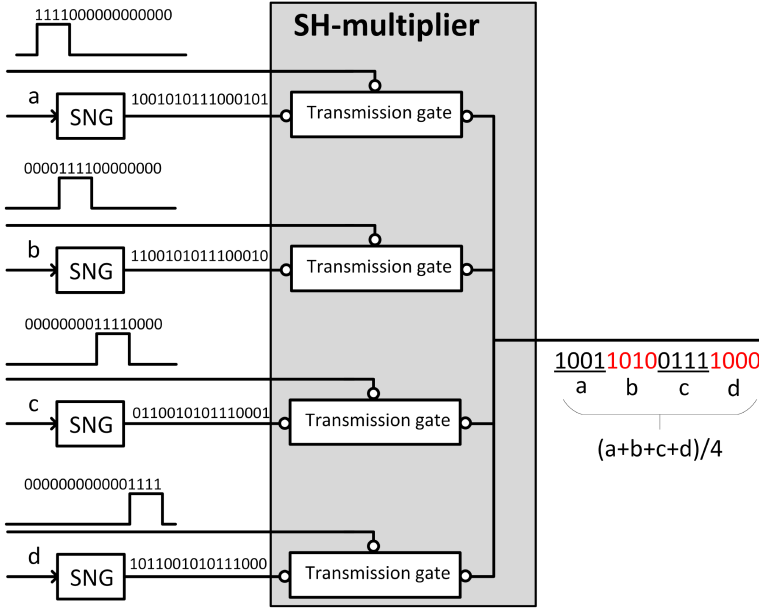


Fig. 11. An example of a 4-input SH-multiplier with 2-bit quantized weights.

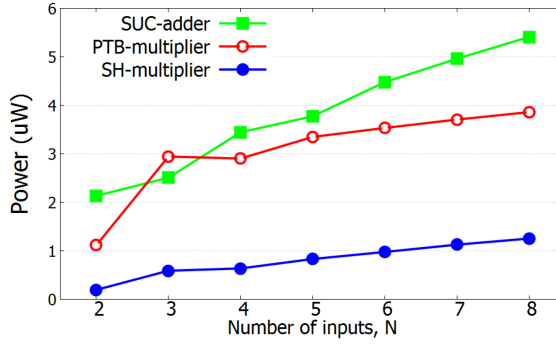


Fig. 12. Average power comparison among the SUC-adder, PTB-multiplier, and SH-multiplier with 3-bit quantization.

quantization has four quantized weight combinations, including $(1/8, 7/8)$, $(2/8, 6/8)$, $(3/8, 5/8)$, and $(4/8, 4/8)$. In terms of power, the results are carried out using a 45nm standard cell library in HSPICE. To get the average power, we simulated all possible input cases and averaged them for each type of numbers of inputs as seen in Figure 12. Obviously, since the transmission gate involves fewer transistors, the SH-multiplier has a lower hardware cost than the PTB-multiplier. Compared to the SUC-adder, the results show that the new proposed SH-multiplier reduces the power by 4.3x to 10.8x when increasing the number of inputs from 2 to 8, respectively. Regarding area, we normalized the area of the SUC-adder and SH-multiplier by counting the number of transistors. As shown in Figure 13, the SH-multiplier can achieve 4.5x to 9x area reduction compared to SUC-adders. The architectures of the SH-multiplier in Figure 11 and the SUC-adder in Figure 8 can be regarded as 4-input architectures for 3-bit quantization. According to the simulation results,

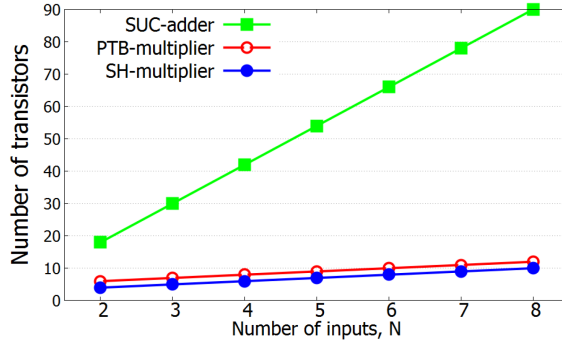


Fig. 13. Area comparison among the SUC-adder, PTB-multiplier, and SH-multiplier with different numbers of inputs for 3-bit quantization.

compared to the architecture (SUC-adder) in Figure 8, the architecture (SH-multiplier) in Figure 11 achieves 2.59x power reduction on average while reducing area about 10%.

The error rates of the proposed multipliers are investigated in the following section. As mentioned earlier, the SUC-adder and SH-multiplier perform the same function, and thus they will achieve the same error rates. Compared to the traditional unipolar adder (only OR gate), as seen in Table 1, our SH-multiplier and SUC-adder have much better accuracy than the OR-based adders. The analysis is given in Section 4.3. In addition, compared to previous stochastic neural network implementations [12, 15, 23], the SH-multiplier can reduce the number of inputs of parallel counters because it simply implements multi-input addition with several AND and OR gates. Therefore, the stochastic quantized neural networks with the SH-multiplier will substantially reduce the hardware cost compared to previous works.

4.3 Stochastic Quantized Weight Generator

In this section, we mainly focus on the stochastic quantized weight generator. As discussed in Section 4.1, quantized weights have 2^n non-zero values (n is the quantization level) in SC. Since each non-zero value needs to be encoded to interleaved bit streams for SUC-adders and SH-multipliers, for each non-zero value $i/2^n$ ($1 \leq i \leq 2^n$), there are $\lfloor 2^n/i \rfloor$ types of shifted unary bit streams. Therefore, it needs $\sum_{i=1}^{2^n} \lfloor 2^n/i \rfloor$ quantized weight generators used in the whole neural networks.

The design of a 4-bit quantized weight generator, which is basically a 4-bit binary counter with parallel load, is shown in Figure 14. This generator can be used in generating bit streams with a period of 16. Initialization of the counter is done by parallel loading of the desired initial value ($L_3L_2L_1L_0$) to the flip-flops. This quantized weight generator can be used in generating bit streams of different levels of quantized weights and different phases of 1's. The Synopsis Design Compiler as our synthesis tool will be responsible for hardware implementation of this weight generator.

To investigate accuracy of the shifted unary bit streams, we use the operations of $(a + b + c + d)/4$ and $(a + b + c + d + e + f + g + h)/8$ with 3-bit quantized weights. Then, we compare our generator to LFSRs by the mean absolute error (MAE). We further compare conventional OR-based adders to our design. In the implementation, all inputs are encoded using LFSR-based SNGs, and the coefficients ($1/4$ and $1/8$) are produced by different methods. Five configurations are compared: (i) SUC-adder/SH-multiplier, in which the same coefficient values are generated by the same quantized weight generator; (ii) parallel adder, in which the same coefficient values are generated by the same LFSR; (iii) parallel adder, in which the same coefficient values are generated by different

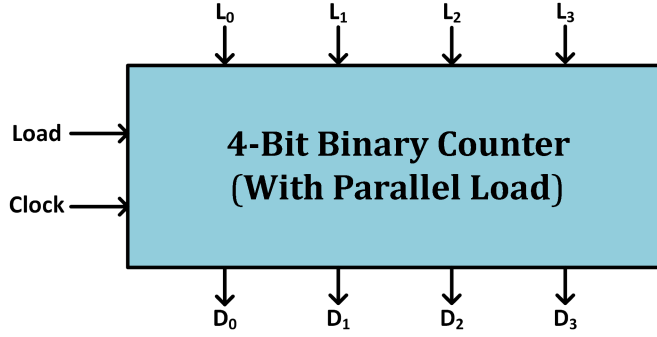


Fig. 14. The structure of a 4-bit quantized weight generator for generating bit streams with a period of 16. A binary value and $D_3D_2D_1D_0$ go through a comparator in each cycle to generate 1 bit of the bit stream corresponding to the target binary value.

Table 1. MAE Comparison Between the Quantized Weight Generator and LFSRs

Bit Length	16	32	64	128
Operation	$(a + b + c + d)/4$			
(i) Quantized weight generator	7.74%	5.17%	3.61%	2.50%
(ii) Same LFSR	9.30%	6.12%	4.00%	2.75%
(iii) Different LFSRs	7.82%	5.33%	3.64%	2.53%
(iv) OR with same LFSR	26.9%	26.8%	26.7%	26.7%
(v) OR with different LFSRs	11.8%	10.2%	9.40%	8.97%
Operation	$(a + b + c + d + e + f + g + h)/8$			
(i) Quantized weight generator	8.16%	5.60%	3.88%	2.69%
(ii) Same LFSR	11.88%	7.80%	5.03%	3.28%
(iii) Different LFSRs	8.50%	5.63%	3.94%	2.75%
(iv) OR with same LFSR	37.6%	37.7%	37.4%	37.5%
(v) OR with different LFSRs	12.5%	11.0%	10.3%	9.91%

LFSRs; (iv) OR adder, in which the same coefficient values are generated by the same LFSR; and (v) OR adder, in which the same coefficient values are generated by different LFSRs.

As seen in Table 1, due to the correlation between bit streams, the OR-based adder with the same LFSR gets the worst MAE results, and the OR-based adder with different LFSRs is worse than our design. In addition, our quantized weight generator has shown even better MAE results with different bit lengths than the parallel counter implementations. Therefore, the bit streams generated by our quantized weight generators can obtain similar or even more accurate results compared to the conventional LFSR-based implementations. The details of hardware comparison of generators will be discussed in Section 5.2.

4.4 Stochastic Neuron Implementation

A single neuron is the basic unit in neural networks. Each neuron consists of a matrix multiplication and an activation function. The structure of a stochastic neuron is shown in Figure 15. The inputs of neurons come from bit streams corresponding to input image pixels generated by SNGs, or outputs of previous layers. Since image pixel inputs and outputs of previous layers (outputs of

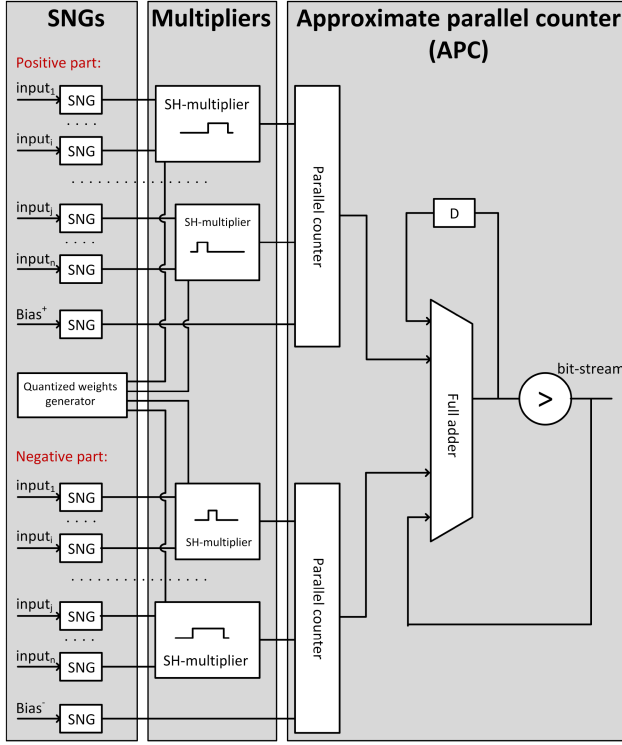


Fig. 15. The structure of a stochastic neuron with matrix multiplication.

sigmoid function) are always in the range of $[0, 1]$, the signs of products of inputs and quantized weights are determined by the quantized weights. Therefore, the number of positive and negative products are clearly known when the classifiers have been trained.

In our proposed neuron, inputs are separated into positive and negative groups based on the signs of their quantized weights and are ANDed with the quantized weights to compute their products. Then, we group some of the products going to the same SUC-adder and SH-multiplier. Since the number of quantized weights is determined, we can minimize the number of SUC-adders/SH-multipliers by adding as many products as possible in one SUC-adder/SH-multiplier. For example, we can group two products with weights of $1/4$ and $3/4$, or three products with weights of $1/4$, $1/4$, and $2/4$. After that, partial addition results and bias are assigned to parallel counter to sum bit streams up. $bias^+$ and $bias^-$ in the positive and negative parts are extra constant inputs, which are obtained from training process as constant values. Finally, the full adder will compute the subtraction of the positive and negative part and go through a comparator to generate a bit stream that is the input of next layer.

The sigmoid function is used as the activation function in the RBM. In this work, we use the approximate format of the sigmoid function proposed by Li et al. [19]. The basic idea is to use the Taylor expansion of the sigmoid function: $\text{sigmoid}(x) = 1/2 + x/4 - x^3/48 \dots$. Since the values in SC are in the range $[0, 1]$ or $[-1, 1]$, the terms with high degrees can be ignored, and the approximate sigmoid function becomes $\frac{x+2}{4}$, and it is automatically achieved in the proposed neuron. As discussed in Li et al. [19], the weights are scaled down by four and the trained positive biases are added by $1/2$.

Table 2. Recognition Error Rate Comparisons of RBM Classifiers With Different Bit Lengths

	Quantization	2 Bits	3 Bits	4 Bits
Conventional Binary	0.98%			
Quantized Binary		1.44%	1.18%	0.99%
	Bit length			
Prior work [15]	16	4.29%	7.68%	15.59%
	32	2.86%	2.80%	3.36%
	64	2.52%	2.24%	2.08%
	128	1.92%	2.03%	1.80%
Our method (SH-multiplier/SUC-adder)	16	2.73%	2.59%	2.39%
	32	2.71%	2.42%	2.05%
	64	2.56%	2.49%	1.85%
	128	2.37%	1.82%	1.79%

5 EXPERIMENTAL RESULTS OF NEURAL NETWORK COMPARISON

In this section, we use an RBM classifier implemented by SC with a configuration of 784-500-1000-10. The MNIST handwritten digit image dataset [14] is used as input images, which consists of 70,000 images. Among the images, 60,000 images are training images and the remaining 10,000 images are used for testing stochastic and conventional neural networks. All weights and coefficients of neural networks are first fully trained and then retrained as described in Section 3.2.

5.1 Recognition Error Rates

We first compare error rates of our stochastic implementation to previous stochastic methods and deterministic RBM implementation. In the stochastic implementation, we compared our work to the stochastic method in Lee et al. [15]. The bit stream lengths are varied from 16 bits to 128 bits.

As seen in Table 2, the quantized binary RBM implementation obtains recognition error rates quite similar to the conventional binary RBM. For the stochastic RBM comparisons, our method obtains slightly higher recognition error rates than the binary implementation. However, compared to the previous work, our implementation achieves much lower recognition error rates at shorter bit stream lengths.

Since a transmission gate is a non-restoring device, it may introduce some noise into the design. To investigate the effect of such noise on the recognition error rate of the proposed design, soft errors [28] are manually injected into the outputs of transmission gates as seen in Figure 10. The injected error rate indicates the probability of flipping the bit of the outputs per cycle. For example, an injected error of 10% indicates that there is a 10% probability to flip the output bit from 0 to 1 or 1 to 0 in each cycle. As seen in Figure 16, we use the 4-bit quantized neural network with 32-bit stream length and inject the soft error from 0% to 10%. With the injected error rate smaller than 6%, the recognition error rate stays lower than 5%. This shows the noise tolerance capacity of the stochastic quantized neural network.

5.2 Hardware Cost

In this section, we discuss the hardware cost of different neural network implementations. We use the Synopsys Design Compiler vH2013.12 to synthesize neural networks with the FreePDK 45nm library [31] and use HSPICE with the same 45nm library for circuit-level evaluation of stochastic implementations. All simulations are based on 3-bit quantized neural networks. The binary

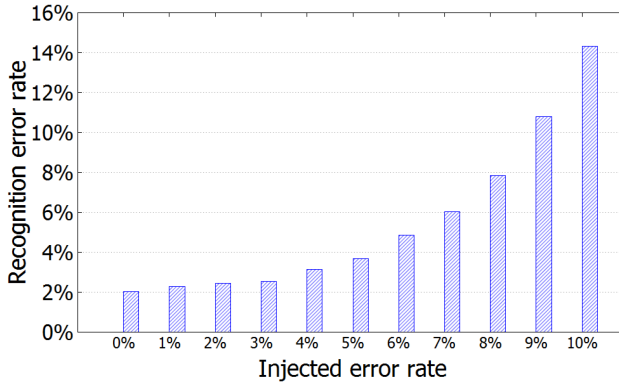


Fig. 16. The performance effect of injecting errors into the transmission gates.

Table 3. Hardware Comparison Between the LFSR and Quantized Weight Generator

Types of SNGs	Area (μm^2)	Power (μW)
4-bit LFSR	81.7	15.2
5-bit LFSR	100.4	18.7
4-bit quantized weight generator	74.6	14.4
5-bit quantized weight generator	99.0	17.2

implementation uses 4-bit fixed-point binary as inputs and 7-bit fixed-point binary as multiplication outputs (the most significant bit (MSB) is the sign bit).

First, we compare hardware cost of the conventional LFSR and our proposed quantized weight generator. As seen in Table 3, our proposed generator has a little lower area and power compared to the conventional LFSR for producing different lengths of bit streams. Thus, our generator does not introduce any extra overhead in the hardware implementations.

There are mainly two types of implementations in current stochastic neural networks. One type [12, 23] is based on bipolar format. It normally encodes input images and weights as bipolar format bit streams. Then, to achieve a neuron including matrix multiplication and activation function, the bit streams first go into parallel counters and then go to finite state machines. Finally, outputs of a neuron keep the bipolar format and are matched to next layers. The other type of stochastic implementations [15, 19] is unipolar based. Its structure is quite similar to ours. The differences between our method and other unipolar-based implementations lie in the stochastic matrix multiplication and quantized weight generator.

In the hardware comparison, we compare our design to the conventional binary implementation, and bipolar and unipolar stochastic methods. As seen in Table 4, compared to previous stochastic implementations (unipolar and bipolar), our architecture obtains about $5.94\times$ and $9.58\times$ reduction in terms of power and energy, respectively. This is because our design significantly decreases the sizes of parallel counters and finally obtains reduction of hardware cost. Moreover, compared to the binary implementation, our approach with 32-bit length derives about $80\times$ and $7.7\times$ less power and energy, respectively. Furthermore, even when we increase the bit length to 128, our design still consumes about $2\times$ less energy compared to the binary implementation.

The area results of the SUC-adder and previous works are based on the RTL-level synthesis report from the Synopsys Design Compiler. The numbers we report for the PTB-multiplier and the

Table 4. Hardware Comparison Between the Stochastic RBM With 32-Bit Length and the Conventional RBM Implementation With SNGs

Neural Networks	Area (mm^2)	Power (mW)	Critical Path (ns)	Energy (nJ)
Binary	42.06	4684.6	5.08	23.80
Stochastic unipolar [15]	4.83	525.0	1.99	33.48
Stochastic bipolar [12, 23]	6.67	872.0	3.71	103.67
SUC-adder based	2.21	79.9	1.64	4.21
PTB-multiplier	—	69.67	1.65	3.68
SH-multiplier based	—	58.55	1.65	3.09

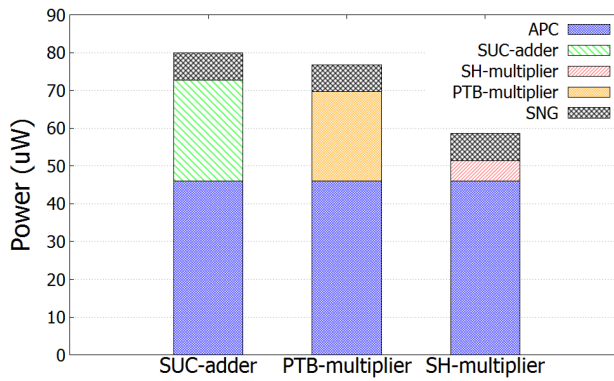


Fig. 17. Power comparison among the SUC-adder, PTB-multiplier, and SH-multiplier stochastic neural network implementations in detail.

SH-multiplier are based on the circuit-level implementation of the designs in HSPICE. To evaluate the areas of such design and have an accurate estimation, the layout of the design needs to be drawn. For a fair comparison, the area comparison among the SUC-adder, the PTB-multiplier, and the SH-multiplier is shown in Figure 13 based on the number of transistors.

We also compared SH-multiplier-based stochastic neural network implementations to SUC-adder and PTB-multiplier-based stochastic implementations in detail. As shown in Figure 15, the stochastic neural network mainly consists of three parts: SNGs, multipliers, and APCs. We analyze the SUC-adder, PTB-multiplier, and SH-multiplier-based stochastic neural network implementations with those three parts. As indicated in Figures 17 and 18, the SH-multiplier, PTB-multiplier, and SUC-adder-based implementations have the same power and area cost of the APC and SNG parts since they use the same APC architecture [19, 23] and the same stochastic quantized weight generators as mentioned in Section 4. The area of SNGs only occupies around 10% of the hardware cost. This is because in the quantized stochastic design, all weights are quantized values and we only need $\sum_{i=1}^{2^n} \lfloor 2^n/i \rfloor$ generators for all trained weights. For those floating-point inputs (only at the first layer), since the weights select the different phases of the inputs as introduced in Section 4.1, the quantized multiplication and addition in this article do not face the correlation issue. Therefore, we can use only one RNG for all input values. As a result, the effective area of SNGs is smaller than that of the multipliers.

For the multiplier part, the SH-multiplier-based implementation has $4.92\times$ power reduction and $6.47\times$ area reduction compared to the SUC-adder-based implementation. Compared to the

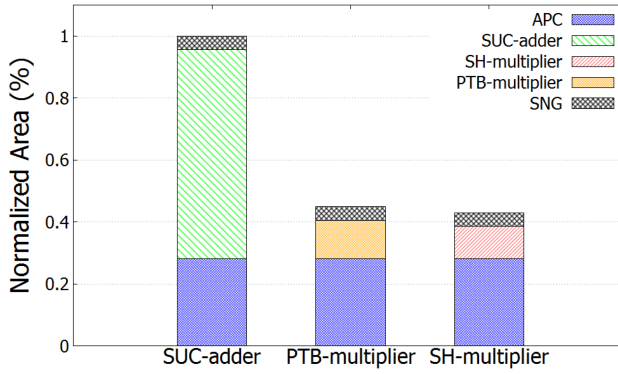


Fig. 18. Area comparison among the SUC-adder, PTB-multiplier, and SH-multiplier stochastic neural network implementations in detail.

PTB-multiplier, the SH-multiplier achieves $4.35\times$ power reduction and 19% area reduction. Moreover, according to Figures 17 and 18, the multiplier part is the major part of the whole neural network implementation and occupies 33.3% of the total power consumption and 67.5% of the total area cost for the SUC-based implementation. Therefore, the proposed SH-multiplier-based implementation is capable of reducing the occupancy to about 6% and 13% in terms of power and area, respectively. In summary, the new SH-multiplier-based implementation achieves $2.33\times$ area reduction and reduces about 36% the power and energy cost compared to the SUC-adder-based implementation.

6 CONCLUSION

In this article, we propose a new stochastic architecture for quantized neural networks. The quantized neural networks are retrained by different quantization levels and obtain recognition error rates similar to floating-point neural networks. We propose a new multiplier—the SH-multiplier—which is capable of summing several product bit streams without losing precision. The SH-multiplier can reduce the number of inputs of parallel counters and hence decrease the hardware cost of the neural networks. As shown in experimental results, in terms of area, power, and energy, our approach with 32-bit length derives at least $4\times$, $9\times$, and $10\times$ reduction compared to previous stochastic implementations and obtains $80\times$ power reduction and $7.7\times$ energy reduction compared to the corresponding binary implementation while maintaining slightly higher recognition error rates than the binary implementation. Compared to our prior SUC-based implementation, the SH-multiplier achieves $2.65\times$ area reduction and about 36% power and energy reduction.

REFERENCES

- [1] Armin Alaghi and John P. Hayes. 2013. Exploiting correlation in stochastic circuit design. In *Proceedings of the IEEE 31st International Conference on Computer Design (ICCD'13)*. IEEE, Los Alamitos, CA, 39–46.
- [2] Armin Alaghi and John P. Hayes. 2013. Survey of stochastic computing. *ACM Transactions on Embedded Computing Systems* 12, 2s (2013), Article 92.
- [3] Bradley D. Brown and Howard C. Card. 2001. Stochastic neural computation. I. Computational elements. *IEEE Transactions on Computers* 50, 9 (2001), 891–905.
- [4] J. A. Dickson, R. D. McLeod, and H. C. Card. 1993. Stochastic arithmetic implementations of neural networks with in situ learning. In *Proceedings of the IEEE International Conference on Neural Networks*. IEEE, Los Alamitos, CA, 711–716.
- [5] Brian R. Gaines. 1969. Stochastic computing systems. *Advances in Information Systems Science* 2, 2 (1969), 37–172.
- [6] Song Han, Huizi Mao, and William J. Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. arXiv:1510.00149.

- [7] Geoffrey E. Hinton. 2012. A practical guide to training restricted Boltzmann machines. In *Neural Networks: Tricks of the Trade*. Springer, 599–619.
- [8] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *Journal of Machine Learning Research* 18, 1 (2017), 6869–6898.
- [9] Kyu Yeon Hwang and Wonyong Sung. 2014. Fixed-point feedforward deep neural network design using weights+ 1, 0, and- 1. In *Proceedings of the IEEE Workshop on Signal Processing Systems (SiPS'14)*. IEEE, Los Alamitos, CA, 1–6.
- [10] Devon Jenson and Marc Riedel. 2016. A deterministic approach to stochastic computation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD'16)*. IEEE, Los Alamitos, CA, 1–8.
- [11] Seul Jung and Sung Su Kim. 2007. Hardware implementation of a real-time neural network controller with a DSP and an FPGA for nonlinear systems. *IEEE Transactions on Industrial Electronics* 54, 1 (2007), 265–271.
- [12] Kyoung Hoon Kim, Jungki Kim, Joonsang Yu, Jungwoo Seo, Jongeun Lee, and Kiyoun Choi. 2016. Dynamic energy-accuracy trade-off using stochastic computing in deep neural networks. In *Proceedings of the 53rd Annual Design Automation Conference*. ACM, New York, NY, 124.
- [13] Kyoung Hoon Kim, Jongeun Lee, and Kiyoun Choi. 2016. An energy-efficient random number generator for stochastic circuits. In *Proceedings of the 2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC'16)*. IEEE, Los Alamitos, CA, 256–261.
- [14] Yann LeCun and Corinna Cortes. 2010. Database of Handwritten Digits. Retrieved March 11, 2019 from <http://yann.lecun.com/exdb/mnist>.
- [15] Vincent T. Lee, Armin Alaghi, John P. Hayes, Visvesh Sathe, and Luis Ceze. 2017. Energy-efficient hybrid stochastic-binary neural networks for near-sensor computing. In *Proceedings of the 2017 Design, Automation, and Test in Europe Conference and Exhibition (DATE'17)*. IEEE, Los Alamitos, CA, 13–18.
- [16] Bingzhe Li, M. Hassan Najafi, and David J. Lilja. 2015. An FPGA implementation of a restricted Boltzmann machine classifier using stochastic bit streams. In *Proceedings of the 2015 IEEE 26th International Conference on Application-Specific Systems, Architectures, and Processors (ASAP'15)*. IEEE, Los Alamitos, CA, 68–69.
- [17] Bingzhe Li, M. Hassan Najafi, and David J. Lilja. 2016. Using stochastic computing to reduce the hardware requirements for a restricted Boltzmann machine classifier. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, New York, NY, 36–41.
- [18] B. Li, M. H. Najafi, B. Yuan, and D. J. Lilja. 2018. Quantized neural networks with new stochastic multipliers. In *Proceedings of the 2018 19th International Symposium on Quality Electronic Design (ISQED'18)*. 376–382. DOI : <https://doi.org/10.1109/ISQED.2018.8357316>
- [19] Bingzhe Li, Yaobin Qin, Bo Yuan, and David J. Lilja. 2017. Neural network classifiers using stochastic computing with a hardware-oriented approximate activation function. In *Proceedings of the 2017 IEEE 35th International Conference on Computer Design (ICCD'17)*. IEEE, Los Alamitos, CA, 97–104.
- [20] Ji Li, Zihao Yuan, Zhe Li, Caiwen Ding, Ao Ren, Qinru Qiu, Jeffrey Draper, et al. 2017. Hardware-driven nonlinear activation for stochastic computing based deep convolutional neural networks. arXiv:1703.04135.
- [21] Peng Li, David J. Lilja, Weikang Qian, Kia Bazargan, and Marc D. Riedel. 2014. Computation on stochastic bit streams digital image processing case studies. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 22, 3 (2014), 449–462.
- [22] Peng Li, David J. Lilja, Weikang Qian, Marc D. Riedel, and Kia Bazargan. 2012. Logical computation on stochastic bit streams with linear finite-state machines. *IEEE Transactions on Computers* 63, 6(2012), 1474–1486.
- [23] Zhe Li, Ao Ren, Ji Li, Qinru Qiu, Yan zhi Wang, and Bo Yuan. 2016. DSCNN: Hardware-oriented optimization for stochastic computing based deep convolutional neural networks. In *Proceedings of the IEEE 34th International Conference on Computer Design (ICCD'16)*. IEEE, Los Alamitos, CA, 678–681.
- [24] Siting Liu and Jie Han. 2017. Energy efficient stochastic computing with Sobol sequences. In *Proceedings of the 2017 Design, Automation, and Test in Europe Conference and Exhibition (DATE'17)*. IEEE, Los Alamitos, CA, 650–653.
- [25] Ankit Mondal and Ankur Srivastava. 2017. Power optimizations in MTJ-based neural networks through stochastic computing. In *Proceedings of the IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED'17)*. IEEE, Los Alamitos, CA, 1–6.
- [26] M. Hassan Najafi, Shiva Jamali-Zavareh, David J. Lilja, Marc D. Riedel, Kia Bazargan, and Ramesh Harjani. 2017. Time-encoded values for highly efficient stochastic circuits. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 5 (2017), 1644–1657.
- [27] Amos R. Omondi and Jagath Chandana Rajapakse. 2006. *FPGA Implementations of Neural Networks*. Vol. 365. Springer.
- [28] Weikang Qian, Xin Li, Marc D. Riedel, Kia Bazargan, and David J. Lilja. 2011. An architecture for fault-tolerant computation with stochastic logic. *IEEE Transactions on Computers* 60, 1 (2011), 93–105.
- [29] Weikang Qian and Marc D. Riedel. 2010. Synthesizing logical computation on stochastic bit streams. http://www.mriedel.ece.umn.edu/wiki/images/6/64/Qian_Riedel_Synthesizing_Logical_Computation_on_Stochastic_Bit_Streams.pdf.

- [30] Ruslan Salakhutdinov and Geoffrey Hinton. 2009. Deep Boltzmann machines. In *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics*. 448–455.
- [31] James E. Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, et al. 2007. FreePDK: An open-source variation-aware design kit. In *Proceedings of the IEEE International Conference on Microelectronic Systems Education (MSE'07)*. IEEE, Los Alamitos, CA, 173–174.
- [32] Rangharajan Venkatesan, Swagath Venkataramani, Xuanyao Fong, Kaushik Roy, and Anand Raghunathan. 2015. Spintastic: Spin-based stochastic logic for energy-efficient computing. In *Proceedings of the Design, Automation, and Test in Europe Conference and Exhibition (DATE'15)*. IEEE, Los Alamitos, CA, 1575–1578.
- [33] Meng Yang, Bingzhe Li, David J. Lilja, Bo Yuan, and Weikang Qian. 2018. Towards theoretical cost limit of stochastic number generators for stochastic computing. In *Proceedings of the 2018 IEEE Computer Society Annual Symposium on VLSI (ISVLSI'18)*. IEEE, Los Alamitos, CA, 154–159.
- [34] Dong Yu, Frank Seide, Gang Li, and Li Deng. 2012. Exploiting sparseness in deep neural networks for large vocabulary speech recognition. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP'12)*. IEEE, Los Alamitos, CA, 4409–4412.
- [35] Reto Zimmermann and Wolfgang Fichtner. 1997. Low-power logic styles: CMOS versus pass-transistor logic. *IEEE Journal of Solid-State Circuits* 32, 7 (1997), 1079–1090.

Received June 2018; revised November 2018; accepted January 2019